

STRUCT MEMORY-R1: Long-Horizon Conversational Context Access and Updates over Tree-Structured Memory with Reinforcement Learning

Anonymous Author(s)

Affiliation

Address

email

Abstract

Memory-augmented Large Language Models (LLMs) maintain long-horizon memory over conversational history to provide relevant context for future interactions. Conversational information has natural event and topic based structure that a tree can capture better than a flat representation. However, deciding how to traverse relevant tree branches to access memory and where in the hierarchy new facts belong to update memory becomes substantially harder than with flat structure. Reinforcement learning (RL) with Group Relative Policy Optimization (GRPO) has been shown to improve reasoning ability over complex multi-step trajectories. Therefore, we propose STRUCT MEMORY-R1, a framework that constructs and maintains a tree-structured memory from raw conversations via reinforcement learning with a GRPO-style reward. The framework includes three components: a *Memory Manager* for updating the tree, a *Retrieve Agent* for accessing the tree, and an *Answer Agent* for downstream question answering. On the LoCoMo benchmark, STRUCT MEMORY-R1 achieves the highest overall F1 and LLM-judge accuracy among baseline methods, with particularly large gains on multi-hop (+8.6 F1) and temporal (+14.1 F1) questions over the flat-memory RL baseline.

1 Introduction

Memory-augmented Large Language Models (LLMs) maintain long-horizon memory over conversational history to provide relevant context for future interactions [10, 17, 1], which enables conversational agents to access information beyond the fixed context window. Most existing memory systems represent memories as flat collections and rely on similarity-based retrieval [7, 6, 12], treating all entries as independent and ignoring the inherent relationships among them (Figure 1).

Conversational information has natural event- and topic-based structure that a tree can capture better than a flat representation. Discussions are organized by topics that span multiple events and sessions and individual utterances exist within these groupings. Several prior works have explored tree-structured memory representations, including MemTree [13], RAPTOR [14], and MemWalker [2]. However, these methods rely on heuristic or LLM-prompted approaches to construct and traverse memory hierarchies.

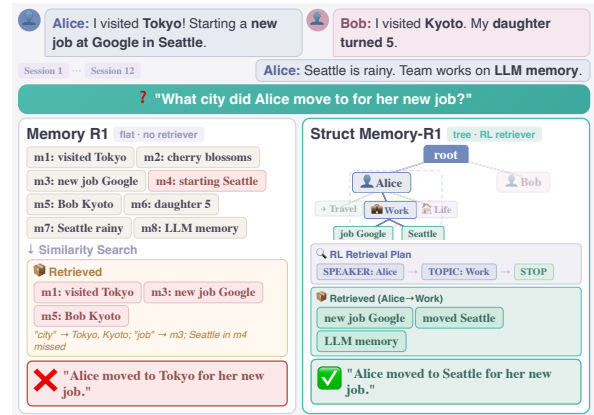


Figure 1: **Memory R1** (left) uses flat similarity search, retrieving “Tokyo” for “city” but missing the job–Seattle link. **Struct Memory-R1** (right) navigates Alice → Work in the tree, correctly finding “Seattle.”

Deciding how to traverse relevant tree branches to access memory and where in the hierarchy new facts belong to update memory is substantially harder than with flat structures. Reinforcement learning (RL) with Group Relative Policy Optimization (GRPO) [15], a policy optimization objective that computes advantages from group-sampled rewards, has been shown to improve reasoning over complex multi-step trajectories. Prior work has already demonstrated its effectiveness for search [4] and flat memory [19]. This approach transfers naturally to structured memory, since the quality of intermediate memory access and update decisions can be evaluated through a verifiable downstream answer to a question.

We therefore propose STRUCT MEMORY-R1, a framework that maintains tree-structured memory from raw conversation and uses RL with GRPO-style question-answering reward to train three coordinated components (Figure 2):

- A **Memory Manager** that receives a new turn and the current memory, then incrementally constructs and updates the memory tree through structural placement of new facts.
- A **Retrieve Agent** that operates over the induced tree and emits an explicit search plan, selecting branches and leaves rather than relying on global flat retrieval.
- An **Answer Agent** that reads retrieved evidence and produces the final answer.

In summary, the paper makes the following contributions:

- We argue that conversational memory has a natural event- and topic-based structure that is better captured by a hierarchical tree representation than by flat memory stores.
- We propose STRUCT MEMORY-R1, a three-component framework that maintains tree-structured memory from raw conversations and learns structure-aware access and update policies via GRPO-based reinforcement learning.
- We introduce STRUCT-LOCOMO, a tree-structured version of the LoCoMo benchmark, specifically designed to support learning and evaluation over hierarchical memory representations.
- We provide a faithful re-implementation of the Memory-R1 framework, establishing a strong and reproducible RL-based baseline for comparison under a unified experimental setting.
- We evaluate our approach on the LoCoMo benchmark [10], demonstrating that tree-structured memory yields substantial gains on multi-hop (+9.0 F1) and temporal (+13.8 F1) questions, while achieving the highest overall F1 among non-oracle methods.

2 Problem Definition

Let π_θ denote an LLM parameterized by θ that generates text autoregressively: given an input sequence x , the model produces an output $y \sim \pi_\theta(\cdot | x)$.

A *memory state* $M_t \in \mathcal{M}$ is a persistent data structure that accumulates information across turns, where $\mathcal{M}$ denotes the space of all valid memory configurations. The initial memory M_0 may be empty or seeded from prior context.

An *interaction* is a temporally ordered sequence of turns between a user and a memory-augmented agent:

$$\mathcal{I} = ((x_1, M_1, y_1), (x_2, M_2, y_2), \dots, (x_T, M_T, y_T)), \quad (1)$$

where at each turn t ($1 \leq t \leq T$): the user provides an input $x_t \in \mathcal{X}$ (a query, instruction, or follow-up); the agent observes x_t together with the current memory M_t ; and the agent produces a response y_t while updating memory to M_{t+1} . Specifically, each turn involves three operations:

Memory Update. The agent incorporates new information from the current utterance into persistent memory:

$$M_{t+1} = f_{\text{write}}(x_t, M_t).$$

Memory Access. When the input contains a question q_t , the agent retrieves a relevant subset of evidence from memory:

$$R_t = f_{\text{read}}(q_t, M_t).$$

Answer Generation. The agent produces a response conditioned on the question and retrieved evidence:

$$y_t \sim \pi_\theta(\cdot | q_t, R_t).$$

The full system is characterized by the memory space $\mathcal{M}$, the update function f_{write} , the access function f_{read} , and the generation model π_θ . Different instantiations of these components give rise to different memory-augmented

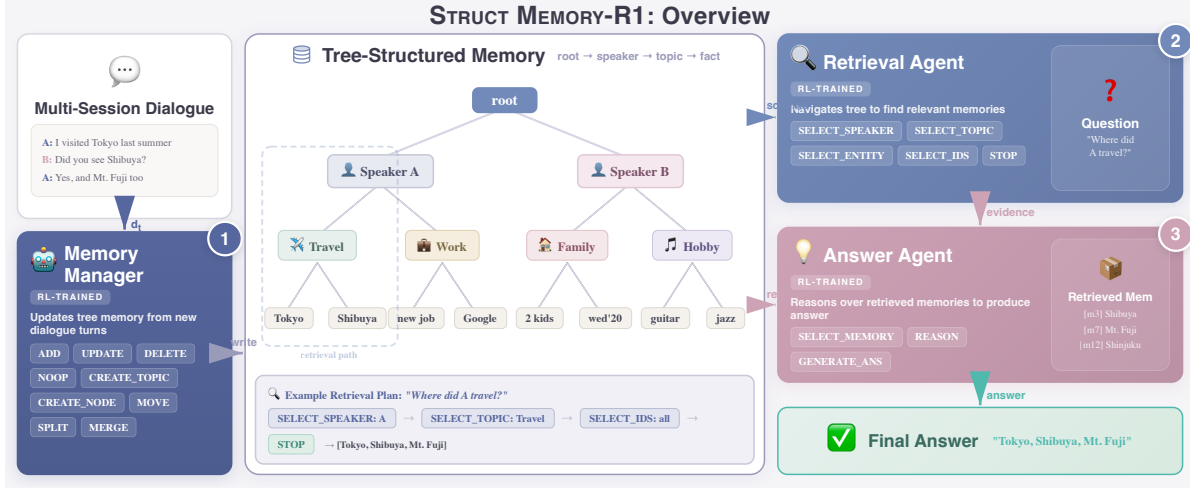


Figure 2: Overview of STRUCT MEMORY-R1. The Memory Manager induces and updates a tree-structured memory from raw conversation. The Retrieve Agent emits a search plan over the tree to select relevant evidence. The Answer Agent produces the final response. All three components are trained with RL using GRPO from downstream question-answering reward.

LLM architectures. In this work, we instantiate $\mathcal{M}$ as tree-structured memory and train all three components with reinforcement learning, as described in Section 3.

3 Methodology

3.1 Overview

As illustrated in Figure 2, STRUCT MEMORY-R1 instantiates the memory-augmented LLM framework defined in section 2 with three learned components: a Memory Manager π_{θ_M} (bottom left), a Retrieve Agent π_{θ_R} (top right), and an Answer Agent π_{θ_A} (bottom right). Each component is parameterized by a language model and trained using Group Relative Policy Optimization (GRPO) [15], with supervision derived from downstream question-answering rewards.

Training proceeds in stages: the Answer Agent is trained first, then frozen while the Retrieve Agent and Memory Manager are trained in sequence. This staged approach ensures stable reward signals at each phase.

3.2 Memory Bank and Tree Structure

The memory bank $\mathcal{T}_t = (V_t, E_t)$ is a rooted tree that is constructed incrementally from raw conversation. Each node $v \in V_t$ resides at a level $\ell(v) \in \{0, 1, \dots, L\}$ and carries content $c(v)$. The tree is structured such that:

- Level 0: the root, representing the full conversation;
- Levels 1 to $L-1$: internal nodes representing hierarchical groupings (e.g., people, topics, events);
- Level L : leaf nodes, each containing an individual memory entry m_i .

Each internal node v maintains a summary $s(v)$ that aggregates information from its children $\text{Ch}(v) = \{u \in V_t : (v, u) \in E_t\}$. The *schema* of the tree, $\text{SCHEMA}(\mathcal{T}_t)$, consists of node identifiers and summaries at each level without exposing full leaf content, providing a compact representation for the retrieval policy.

The tree is serialized as structured JSON for input to language model policies. This serialization preserves the hierarchical relationships while remaining compatible with standard text-based LLM input formats.

3.3 Memory Manager

To properly maintain the tree structure as new dialogues and utterances arrive across multi-session interactions, the system requires a dedicated agent to manage memory operations—namely, determining how to store, insert,

delete, and update both new and existing entries within the current tree-structured memory. The *Memory Manager* observes the current dialogue turn x_t and the current memory tree $\mathcal{T}_t$, and emits a set of operations to update the tree. We restrict the operation space to four stable CRUD operations:

$$\mathcal{O} = \{\text{ADD}, \text{UPDATE}, \text{DELETE}, \text{NONE}\}. \quad (2)$$

Given turn x_t and tree $\mathcal{T}_t$, the manager samples an operation sequence:

$$o_t \sim \pi_{\theta_M}(\cdot \mid x_t, \mathcal{T}_t), \quad (3)$$

and the updated tree is obtained deterministically:

$$\mathcal{T}_{t+1} = \text{APPLY}(\mathcal{T}_t, o_t). \quad (4)$$

The operation o_t is a JSON object specifying which entries to add, modify, or remove, along with their placement in the tree hierarchy. By restricting to CRUD operations rather than arbitrary structural rewrites, we isolate the benefit of structured memory from the complexity of unrestricted topology editing.

Reward. The Memory Manager is trained with outcome-driven reward. After applying the manager’s operations, the downstream pipeline (Retrieve Agent + frozen Answer Agent) processes the corresponding question. The reward is:

$$r_M = \mathcal{R}(y_t, y_t^*) + \lambda_f \cdot \mathbb{I}[\text{valid JSON}], \quad (5)$$

where $\mathcal{R}(y_t, y_t^*)$ measures answer quality and λ_f is a small bonus for well-formed output. This encourages the manager to write memories that are useful for future question answering rather than those matching local heuristics.

3.4 Retrieve Agent

Unlike Memory-R1, where retrieval is treated as conventional dense retrieval over a flat memory bank (i.e., a set of independent documents), tree-structured memory requires a retrieval mechanism that can explicitly traverse the hierarchical structure. Such a mechanism must reason over both the query and the tree schema to navigate relevant branches and locate appropriate memory entries.

The *Retrieve Agent* operates over the memory tree constructed by the Memory Manager. Given a question q and the tree schema $\text{SCHEMA}(\mathcal{T})$, it emits a search plan:

$$z \sim \pi_{\theta_R}(\cdot \mid q, \text{SCHEMA}(\mathcal{T})). \quad (6)$$

The search plan z is a JSON object of the form:

$$z = \{\text{levels} : [\ell_1, \dots, \ell_k], \text{selected_ids} : [id_1, \dots, id_m], \text{stop} : \{0, 1\}\}, \quad (7)$$

where `levels` specifies which branches to expand at each tree level, `selected_ids` names specific leaf nodes to retrieve, and `stop` terminates search. A deterministic executor runs the plan against the tree:

$$R = \text{EXECUTE}(\mathcal{T}, z), \quad (8)$$

returning a set of retrieved memory leaves together with local context (parent summaries, sibling information). This design ensures that retrieval is conditioned on structural locality rather than global similarity scoring.

Reward. The retriever reward combines answer utility with evidence quality:

$$r_R = \mathcal{R}(y_t, y_t^*) + \lambda_{\text{ev}} \cdot \text{HIT}(R, E^*) - \lambda_{\text{cost}} \cdot C(z), \quad (9)$$

where E^* denotes gold evidence nodes (when available), $C(z)$ penalizes search cost, and the primary signal is answer quality under a frozen Answer Agent.

3.5 Answer Agent

The *Answer Agent* follows the same approach as in the Memory-R1 framework. It receives the question q and retrieved memories R , and produces a response:

$$y \sim \pi_{\theta_A}(\cdot \mid q, R). \quad (10)$$

This component is trained to condition on retrieved evidence rather than on the full conversation history. During Memory Manager and Retrieve Agent training, the Answer Agent is frozen and serves as the evaluator in the reward loop, ensuring that improvements in upstream components are measured through their effect on answer quality.

3.6 Training with GRPO

All three components are trained using Group Relative Policy Optimization (GRPO) [15], which avoids the need for a separate value network by computing advantages from a group of sampled outputs. For a given input x and policy π_θ , GRPO samples a group of G outputs $\{o_1, \dots, o_G\}$ and computes normalized advantages:

$$\hat{A}_i = \frac{r_i - \mu(\{r_1, \dots, r_G\})}{\sigma(\{r_1, \dots, r_G\})}, \quad (11)$$

where r_i is the reward for output o_i , and μ, σ denote the group mean and standard deviation. The policy is updated by maximizing:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E}_x \left[\frac{1}{G} \sum_{i=1}^G \min \left(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1-\epsilon, 1+\epsilon) \hat{A}_i \right) - \beta D_{\text{KL}}[\pi_\theta \parallel \pi_{\text{ref}}] \right], \quad (12)$$

where $\rho_i = \pi_\theta(o_i | x) / \pi_{\text{old}}(o_i | x)$ is the importance ratio, ϵ is the clipping parameter, and β controls the KL penalty relative to a reference policy π_{ref} . This objective encourages the model to produce outputs that yield higher rewards relative to the group average while staying close to the reference distribution.

4 Experimental Setup

4.1 Dataset

We evaluate on LoCoMo [10]¹, a long-horizon conversational memory benchmark containing 10 multi-session dialogues with 1,540 non-adversarial QA pairs across four question categories: single-hop, multi-hop, temporal, and open-domain. We exclude the adversarial category following the Memory-R1 evaluation protocol [19]. Following Memory-R1, we use a 1:1:8 train/validation/test split, yielding 152 training, 81 validation, and 1,307 test QA pairs.

Struct-LoCoMo Since LoCoMo provides raw multi-session dialogues without any structured memory representation, we manually convert each sample into a tree-structured memory following the schema used by our framework. Specifically, each dialogue is organized into a four-level hierarchy: Root $\rightarrow$ Conversation $\rightarrow$ Speaker $\rightarrow$ Session $\rightarrow$ Observation, where each leaf node corresponds to an atomic factual observation extracted from a single dialogue turn.

For example, a fact uttered by John in Session 13 is placed at the path Root $>$ Conversation_v1 $>$ John $>$ Session_13 $>$ Observation_1. This conversion is performed by a prompted LLM and subsequently verified by human annotators to ensure structural correctness and factual fidelity. The resulting structured memory serves as the oracle input to our Memory Manager and Retrieve Agent during training and evaluation.

4.2 Model and Hardware

All trainable components use Qwen2.5-3B-Instruct as the base policy model. Experiments are run on a single NVIDIA H200 GPU with 16 CPU cores. This setup supports a single-node staged GRPO training and fixed-answer-agent evaluation. Each agent is trained for 500 steps, and end-to-end training of STRUCT MEMORY-R1 across all three staged components totals approximately 150 GPU hours on this hardware.

We use GRPO as the default policy-optimization method, following Search-R1 and Memory-R1 [4, 19]. For the Memory Manager and Retrieve Agent, prompts are formulated as one-shot JSON generation problems with rule-based reward functions. The Answer Agent is reused directly from Memory-R1 [19], where it is trained to produce short answers conditioned on retrieved memories. During Memory Manager and Retrieve Agent training, the Answer Agent is frozen and used only to compute the reward.

4.3 Baselines

We organize baselines into three categories.

¹The source code and dataset is provided in <https://anonymous.4open.science/r/Struct-MEM-R1-CEB0>, and the checkpoints of the LLM backbone will be released as a Google Drive link in the final submission.

Table 1: Main results on LoCoMo (1:1:8 train/val/test split, 1,307 test QA pairs, adversarial category excluded). F1: token-level F1; B1: BLEU-1; J: LLM-as-judge accuracy (GPT-4o mini). Best non-oracle results per column in **bold**. Oracle results in *italics*.

Method	Overall			Multi-hop			Single-hop			Temporal			Open-domain		
	F1	B1	J	F1	B1	J	F1	B1	J	F1	B1	J	F1	B1	J
Context-based															
No retrieval; answers from raw context or gold evidence															
Full Context	24.2	19.7	45.7	19.7	15.8	40.4	23.9	19.0	45.7	16.2	12.5	37.3	27.0	22.3	48.6
Oracle	<i>31.3</i>	<i>25.7</i>	<i>60.8</i>	<i>34.3</i>	<i>27.9</i>	<i>63.1</i>	<i>31.6</i>	<i>25.9</i>	<i>62.2</i>	<i>35.4</i>	<i>29.7</i>	<i>65.2</i>	<i>29.5</i>	<i>24.3</i>	<i>59.0</i>
Retrieval-based															
Heuristic or similarity-based retrieval; no RL training															
RAG (BM25)	29.9	25.3	49.8	19.0	14.6	28.1	22.6	17.6	30.6	11.2	8.8	25.1	38.7	33.9	67.3
RAG (Embedding)	29.5	24.3	49.6	21.1	15.8	31.5	21.8	17.3	32.4	12.1	8.8	24.2	37.2	31.7	65.1
Semantic XPath	18.5	14.4	39.8	25.9	20.4	53.7	21.3	16.5	45.1	29.8	24.9	49.0	13.4	10.2	31.6
RL-based															
Policy trained with GRPO from downstream QA reward															
Memory R1	27.4	22.7	53.9	22.5	17.4	50.1	27.7	23.6	54.9	20.8	16.5	48.0	30.0	25.2	55.7
STRUCT MEMORY-R1 (Ours)	30.4	25.4	54.4	31.5	26.8	56.2	27.1	21.7	53.7	34.6	28.6	58.6	30.6	25.8	53.4

Context-based. **Full Context** feeds the entire conversation history into the base Qwen2.5-3B-Instruct model without explicit memory or retrieval, relying on the context window alone. **Oracle** uses the gold evidence annotations provided in LoCoMo, representing an upper bound on retrieval quality.

Retrieval-based. **RAG (BM25)** applies BM25 keyword retrieval over a flat memory bank of extracted facts. **RAG (Embedding)** uses text-embedding-3 dense retrieval over the same flat memory bank. **Semantic XPath** [9] generates deterministic XPath-style queries over oracle tree-structured memory.

RL-based. **Memory R1** [19] is a flat-memory system with an RL-trained Memory Manager and Answer Agent, trained on the same base model and the same data split for fair comparison. Memory-R1 does not provide a public codebase; as part of our contributions, we re-implement it from scratch based on the original paper and retrain the backbone from scratch on an H200 GPU, serving as both our primary RL-based baseline and a component of our ablation study.²

4.4 Evaluation Metrics

We report three answer-quality metrics. **F1** is the token-level F1 score between the predicted and gold answers, measuring lexical overlap. **B1** is BLEU-1, a unigram precision metric that captures answer conciseness alongside correctness. **J** is the LLM-as-judge accuracy, where GPT-4o mini evaluates whether the predicted answer is semantically correct given the gold answer.

5 Results

5.1 Main Results

Table 1 reports F1, B1, and J across four question categories on the LoCoMo test set.

STRUCT MEMORY-R1 achieves the highest overall F1 (30.4), B1 (25.4), and J (54.4) among non-oracle methods. The gains are concentrated in multi-hop and temporal reasoning, where tree-structured retrieval enables the model to navigate related facts through shared ancestors rather than scoring each memory independently. On multi-hop questions, our method improves F1 by 9.0 points over Memory R1 (31.5 vs. 22.5) and J by 6.1 points (56.2 vs. 50.1). On temporal questions, the improvement is even larger: 13.8 F1 points and 10.6 J points over Memory R1. These categories require combining information across sessions or reasoning about time, both of which benefit from the hierarchical grouping that tree structure provides.

The picture is different for single-hop and open-domain questions. Memory R1 edges out our method on single-hop (27.7 vs. 27.1 F1, 23.6 vs. 21.7 B1, 54.9 vs. 53.7 J), where flat retrieval is sufficient because the answer resides in a single memory entry and tree navigation adds no benefit. On open-domain questions, RAG (BM25) achieves the

²The source code for our Memory-R1 re-implementation is also provided in the anonymous repository along with the corresponding checkpoints.

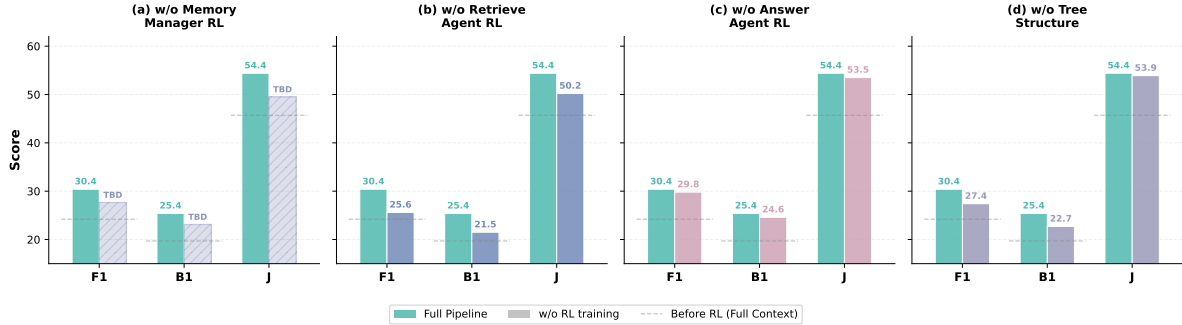


Figure 3: Ablation analysis of STRUCT MEMORY-R1. Each subfigure shows the effect of disabling RL training for one component: (a) Memory Manager, (b) Retrieve Agent, (c) Answer Agent, and (d) replacing tree structure with flat memory. Teal bars show the full pipeline; colored bars show the ablated variant. Grey dashed lines indicate the before-RL baseline (Full Context). Note that “w/o” disables RL fine-tuning for the component, not the component itself. The Memory Manager ablation (hatched bars) is pending completion.

highest F1 (38.7), B1 (33.9), and J (67.3) by a wide margin. Open-domain questions often require broad factual recall that favors high-coverage keyword matching over focused structural navigation. Our method nonetheless outperforms Memory R1 on open-domain F1 (30.6 vs. 30.0), suggesting that tree-organized memory retains some advantage even for general questions.

An interesting pattern appears in the retrieval-based baselines. Semantic XPath achieves substantially higher J than both RAG methods on single-hop (45.1 vs. 30.6/32.4) and multi-hop (53.7 vs. 28.1/31.5) despite comparable or lower F1 and B1. This indicates that structured retrieval surfaces contextually richer evidence that helps the model produce semantically correct answers even when lexical overlap with the gold answer is limited. The same F1–J divergence appears in our method relative to RAG: our J scores are consistently higher than RAG across multi-hop, single-hop, and temporal categories, even when lexical metrics are comparable.

We note that our current results use oracle human-judged structured data for memory construction rather than automatically induced structure. This means the Memory Manager component has not yet been trained end-to-end, and the reported numbers may overestimate the performance achievable with fully automatic structure induction. The ablation study below examines the contribution of each component under this setting.

5.2 Ablation Study

Figure 3 shows the effect of disabling RL training for each component. Disabling Retrieve Agent RL (Figure 3b) causes the largest degradation, dropping F1 from 30.4 to 25.6, B1 from 25.4 to 21.5, and J from 54.4 to 50.2. This confirms that learned tree navigation is the most critical component: without it, the system falls back to heuristic retrieval over tree leaves and loses the ability to plan structured search paths.

Disabling Answer Agent RL (Figure 3c) produces only a modest drop (F1 to 29.8, B1 to 24.6, J to 53.5), suggesting that the base Qwen2.5-3B-Instruct model is already a reasonably capable reader when provided with well-retrieved evidence. The primary bottleneck in our pipeline is retrieval quality, not answer generation.

Replacing tree structure with flat memory (Figure 3d) while keeping all RL components yields numbers that closely match Memory R1 (F1 27.4, B1 22.7, J 53.9), as expected since this ablation is architecturally equivalent to Memory R1. The 3.0-point F1 gap between flat and tree-structured retrieval demonstrates the value of hierarchical memory organization as an inductive bias for the retriever.

The Memory Manager RL ablation (Figure 3a) is ongoing and will quantify the contribution of RL-trained memory writing.

Figure 4 shows the GRPO training reward curves for each component³. The Retrieve Agent (b) achieves a higher final reward than the Answer Agent (a), consistent with the ablation finding that retrieval is the primary bottleneck. Both curves show the characteristic GRPO pattern of rapid initial improvement followed by gradual refinement.

³The Memory Manager is still under training and will be completed before the final submission.

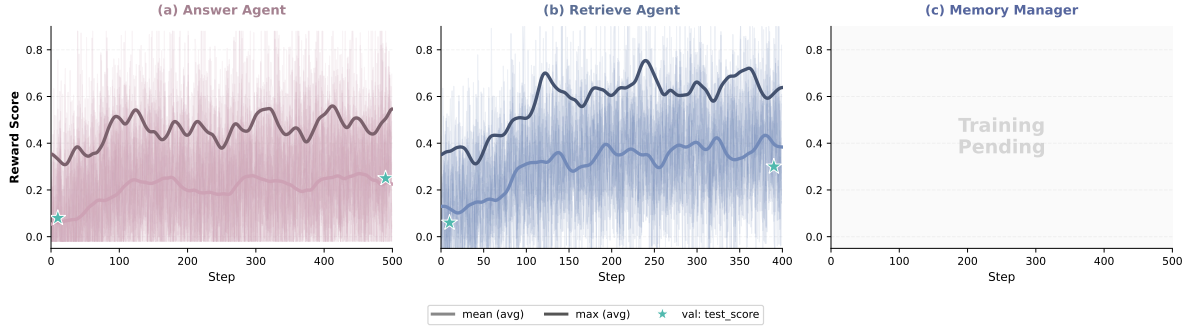


Figure 4: GRPO training reward curves for each component. (a) Answer Agent trained for 500 steps; mean reward increases from 0.08 to 0.25. (b) Retrieve Agent trained for 400 steps; mean reward increases from 0.05 to 0.32, reflecting the larger learning signal from tree navigation. (c) Memory Manager training is pending. Light traces show per-sample rewards; solid lines show smoothed mean and max rewards; stars indicate validation scores.

5.3 Training Dynamics

Figure 4 shows the GRPO training reward curves for each component. Both the Answer Agent (a) and the Retrieve Agent (b) exhibit a characteristic pattern: a rapid initial improvement in the first 50 to 100 steps as the policy quickly learns to produce well-formed outputs, followed by an oscillating plateau where the model refines its behavior more gradually. The Answer Agent mean reward rises from near zero to approximately 0.22 within the first 100 steps, then fluctuates between 0.18 and 0.25 for the remainder of training. The Retrieve Agent shows a similar initial surge but reaches a higher final mean reward (approximately 0.32), consistent with the ablation finding that retrieval is the primary performance bottleneck and thus provides a stronger learning signal. The high variance in per-sample rewards (light traces) reflects the diversity of question difficulty in LoCoMo: some questions are answerable with minimal retrieval while others require precise multi-hop navigation. Memory Manager training (c) is pending and will complete the end-to-end pipeline.

6 Related Work

Flat and Unstructured Memory. Early memory-augmented LLMs retrieve information from flat, unstructured stores via similarity search or rule-based triggers [7, 6]. Agent systems such as Generative Agents [12] popularized the episodic memory stream, a flat chronological log retrieved by recency and relevance. Subsequent work extended this paradigm with explicit memory management: MemGPT [11] introduces working and long-term memory buffers with scheduling policies; MemoryBank [20] proposes a compositional memory controller for lifelong agents; and more recent systems such as Mem0 [3] and A-Mem [18] adopt CRUD-style operations to maintain and evolve flat memory banks through interaction. MemoryOS [5] further treats memory as an operating-system abstraction, providing unified read/write mechanisms across sessions to support long-horizon reasoning. All of these systems represent memory as a flat collection, relying on similarity-based retrieval that treats each entry independently and ignores the relational structure inherent in conversational history—a limitation that STRUCT MEMORY-R1 directly addresses through hierarchical tree organization. These approaches decouple memory access from the model’s reasoning process, leading to over-retrieval and under-utilization of relevant information. A compounding failure mode is the “lost in the middle” phenomenon [8], wherein serializing the full memory state into the context window causes models to neglect centrally positioned entries—motivating selective, reasoning-driven retrieval over both flat and in-context memory baselines.

Structured Memory Access. To address the limitations of flat retrieval, recent work has proposed structured memory representations that expose relational and hierarchical organization to the agent. RAPTOR [14] builds recursive abstractive summaries into a tree for retrieval, but constructs the hierarchy offline over static corpora and does not support incremental updates from live conversation. MemWalker [2] navigates a summary tree interactively to locate relevant context beyond the context limit, but relies on heuristic traversal without a learned access policy. MemTree [13] and Sun and Zeng [16] organize conversational memory into dynamic tree hierarchies that support both read and write operations at multiple abstraction levels, yet both depend on prompted LLMs rather than outcome-driven RL to govern tree construction and traversal. Liu et al. [9] introduce SEMANTICXPATh,

a paradigm in which the LLM generates XPath-style queries over a structured memory tree, enabling precise, path-directed access without flat similarity search; however, the query generation is heuristic and the access policy receives no learning signal from downstream task performance. In contrast, STRUCT MEMORY-R1 learns both tree access and tree update policies end-to-end via RL, directly optimizing hierarchical memory operations for downstream question-answering correctness.

RL-Based Memory and Search. Jin et al. [4] address the analogous limitation in web search by treating search as an explicit reasoning action and training a policy over reasoning–search trajectories via relative reinforcement learning (GRPO). Motivated by this paradigm, Yan et al. [19] extend the idea to flat agent memory, training a two-agent system (Memory Manager + Answer Agent) to perform ADD/UPDATE/DELETE/NOOP operations via RL; while effective on flat stores, this system cannot exploit the hierarchical structure present in tree-organized memory, and its similarity-based retrieval conflates semantically related but structurally distinct entries. STRUCT MEMORY-R1 addresses this gap by extending the RL-over-actions paradigm to structured memory, training agents to learn *where* in the hierarchy to read and write, and *when* structural reorganization is warranted, entirely through end-to-end reinforcement learning over hierarchical memory trajectories. In this work, we observe that memory access is structurally analogous to search, with the key difference that memory operates over an agent’s internal, persistent, and hierarchically organized state [12]. Unlike retrieved web documents, memory entries carry structural provenance—their utility depends not only on semantic relevance but on their position within the memory tree.

7 Conclusion

We introduced STRUCT MEMORY-R1, a reinforcement-learning framework that learns tree-structured conversational memory from raw interaction history. The framework decomposes the memory-augmented generation problem into three components: (1) a Memory Manager for structural updates from tree-based memory, (2) a novel Retrieve Agent that retrieves the correct memory from the tree-based structure conditional on the given questions from the user, and (3) an Answer Agent for evidence-conditioned response generation. Each agent is trained with GRPO from the downstream QA reward.

On the LoCoMo benchmark, STRUCT MEMORY-R1 achieves the highest overall F1 (30.4) and LLM-judge accuracy (54.4) among non-oracle methods, with +9.0 F1 on multi-hop and +13.8 F1 on temporal questions over the flat-memory Memory-R1 baseline. These gains confirm that hierarchical memory organization is most valuable when questions require navigating relationships among facts across sessions. The framework extends the outcome-driven training principle of Search-R1 and Memory-R1 from flat memory to tree-structured access and updates, providing a principled path toward long-horizon conversational agents with learned structured memory.

8 Limitations

The present formulation of STRUCT MEMORY-R1 has several important limitations. **(1) Oracle structure.** The current results rely on oracle human-judged structure for memory construction. While the Memory Manager is designed for end-to-end training, its RL loop has not yet been completed; the reported gains therefore represent a ceiling on what tree structure can provide. **(2) Fixed schema.** The Memory Manager performs CRUD updates under a stable induced schema but does not yet support general tree restructuring (e.g., splitting or merging branches) during training. **(3) Single-pass retrieval.** The Retrieve Agent emits a single search plan rather than acting interactively one level at a time with intermediate observations. Multi-turn tree traversal could improve recall on complex queries. **(4) No hybrid retrieval.** The method emphasizes explicit tree search rather than dense vector retrieval, so it does not yet address the hybrid regime in which semantic similarity and structural routing are both learned. **(5) Single benchmark.** Evaluation is limited to LoCoMo; broader validation on additional long-horizon conversational benchmarks is needed.

References

- [1] Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. Memorybench: A benchmark for memory and continual learning in llm systems. *arXiv preprint arXiv:2510.17281*, 2025.
- [2] Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading. *arXiv preprint arXiv:2310.05029*, 2023.
- [3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

- [4] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- [5] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory OS of AI agent. *arXiv preprint arXiv:2506.06326*, 2025.
- [6] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*, 2020.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [8] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [9] Yifan Simon Liu, Ruifan Wu, Liam Gallagher, Jiazhou Liang, Armin Toroghi, and Scott Sanner. Semantic XPath: Structured agentic memory access for conversational AI. *Under Review*, 2026.
- [10] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*, 2024.
- [11] Charles Packer, Vivian Fang, Shishir G Patil, Kevin Lin, Sarah Wooders, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [12] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- [13] Amin Rezazadeh and Scott Sanner. MemTree: Tree-structured memory for long-horizon conversational agents. *arXiv preprint*, 2024.
- [14] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024.
- [15] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [16] Haoran Sun and Shaoning Zeng. Hierarchical memory for high-efficiency long-term reasoning in LLM agents. *arXiv preprint arXiv:2507.22925*, 2025.
- [17] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2025.
- [18] Wujiang Xu, Kai Mei, Hang Gao, Juntao Tan, Zujie Liang, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [19] Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Kristian Kersting, Jeff Z Pan, Hinrich Schütze, et al. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. *arXiv preprint arXiv:2508.19828*, 2025.
- [20] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024.